WORKSHOP: PROGRAMMING LAB

› Session 5

Strings, lists, tuples and sets

Dr. Ramon Winterhalder

Human-Centered Artificial Intelligence · A.Y. 2026/27



UNIVERSITÀ DEGLI STUDI DI MILANO

INFN MILANO

THE PLAN

Where we are

1 – 2

Ramon Winterhalder

Setup, first programs · variables, types, input

3 – 4

Fatme Hachem

Conditions, try / except · loops and lists

5

Ramon Winterhalder

Today: strings, lists, tuples, sets

6 – 9

Fatme Hachem

Dictionaries, functions, files, libraries

You already know lists and loops. Today: strings are sequences too, a closer look at lists, and two new containers — tuples and sets.

MATERIALS

Where to find the material

myAriel

The official course page: announcements and all material, sessions 1–9.

myariel.unimi.it/user/index.php?id=14344

GitHub

Slides, exercise notebooks and solutions for sessions 1–3. No login needed.

github.com/ramonpeter/programming-lab-hcai-2026

No access to Ariel yet? Use GitHub. Everything for sessions 1-2 & 5 is there.

SESSION 5 · 4 HOURS

Today

0	**Last week's exercises**	4, 5 and 6 from session 4, briefly	20′
1	**Strings are sequences**	indices, slices, quotes and escapes	40′
2	**String methods**	strip, lower, split, join, find …	45′
	Break		15′
3	**Lists, a closer look**	two names for one list · removing safely · break / continue	40′
4	**Tuples and sets**	fixed records · uniqueness · set operations	35′
5	**Mini-project**	todo.py — a tiny command interpreter	40′

00

Last week’s exercises

Exercises 4, 5 and 6 from session 4 — a short recap of the solutions

Exercises 4 and 5: the accumulator

4 · sum of 1 … 100

```
total = 0
for number in range(1, 101):
    total += number
print("Sum:", total)

Sum: 5050
```

Start value = the neutral element. 0 for a sum, 1 for a product — start the product at 0 and the answer is 0, whatever the list.

5 · sum, product, average

```
numbers = [10, 20, 30, 40, 50, 60, 70, 80, 90]
total = 0
product = 1
for x in numbers:
    total += x
    product *= x
average = total / len(numbers)
print(total, product, average)

450 3628800000000000 50.0
```

Exercise 6: keep asking until 5 valid numbers

```
numbers = []
while len(numbers) < 5:
    try:
        prompt = "Number " + str(len(numbers) + 1) + ": "
        num = int(input(prompt))
        numbers.append(num)
    except ValueError:
        print("Invalid input! Please enter an integer.")

print("You entered:", numbers)
for n in numbers:
    if n % 2 == 0:
        print(n, "is even")
    else:
        print(n, "is odd")
```

- The loop condition counts valid numbers, not attempts: a typo costs nothing
- int() fails before append — so a bad input never reaches the list
- try / except inside the loop: one bad input, one message, ask again

Exercise 6 asked for a for loop

```
total = 0
largest = numbers[0]
smallest = numbers[0]
for n in numbers:
    total += n
    if n > largest:
        largest = n
    if n < smallest:
        smallest = n
print(total, largest, smallest)
```

Same result as sum(), max(), min()

- Built-ins are what you use in practice
- The loop is what they do inside — and works for anything they cannot do
- Start largest with the first number, not with 0: with only negative numbers, 0 would win and was never typed

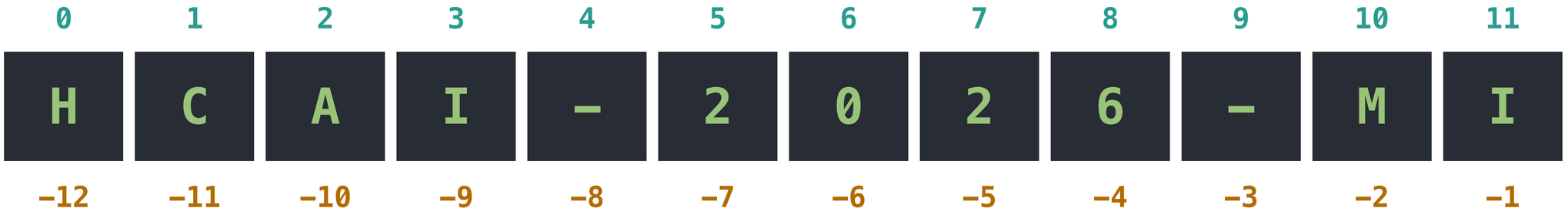
Today's link: a string is a sequence too. for ch in text: works exactly like for n in numbers:.

01

Strings are sequences

Positions, slices, and why you cannot change a single letter

Every character has a position




```
code = "HCAI-2026-MI"
code[0]       # 'H'
code[-1]      # 'I'
code[5:9]     # '2026'
```

```
code[:4]      # 'HCAI'
code[-2:]     # 'MI'
code[::-1]    # 'IM-6202-IACH'
code[10:20]   # 'MI' — no error
```

s[start:stop:step] — start is included, stop is not, so code[5:9] has 9 − 5 = 4 characters. A slice past the end is fine; a single index past the end is an IndexError.

Strings cannot change

```
word = "lab"
word[0] = "L"

TypeError: 'str' object does not support item
assignment
```

```
word = "L" + word[1:]   # a new string
print(word)

Lab
```

Lists can be changed in place. Strings cannot: you build a new string and move the name to it.

Special characters

`\n`	new line
`\t`	tab
`\" \'`	a quote inside quotes
`\\`	a backslash
`""" … """`	text over several lines

Windows paths: "C:\new\table" contains a new line and a tab. Write "C:\\new\\table" or use forward slashes.

EXERCISE A ⏱ 25 min · notebook lab05-exercises.ipynb

A1	core	Write the index row of "HCAI-2026-MI" on paper first. Then predict nine slices and run.
A2	core	date = "2026-11-02": year, month, day by slicing; print 02.11.2026 and the next month.
A3	core	word[0] = "L" fails. Make "Lab" from "L" and a slice of word.
A4	core	Print a quote inside quotes, two lines with one print, a tab table, a triple-quoted string.
A5	stretch	Hide all but the last 4 digits: **********1234 — for any length. Then groups of 4.
A6	challenge	code[5:2], code[5:2:-1], code[-1:-4:-1], code[::-3] — why is the first empty?

Write the index row first — the ones who do get the slices right.

02

String methods

Clean, search, split and join text — always getting a new string back

STRING METHODS

The methods worth knowing

`s.strip()`	remove spaces at both ends	`"  Lab " → "Lab"`
`s.lower()  s.upper()`	change case	`"Lab" → "lab"`
`s.replace(a, b)`	every a becomes b	`"lab" → "Lab"`
`s.find(x)`	first position of x, or -1	`"lab".find("b") → 2`
`s.count(x)`	how often x appears	`"a-b-c".count("-") → 2`
`s.startswith(x)`	True / False	`also endswith`
`s.isdigit()`	only digits?	`"20.5" → False`
`s.split(sep)`	cut into a list of strings	`"a,b" → ['a', 'b']`

Everything is case-sensitive: "Lab".find("lab") is -1.

STRING METHODS

A method gives you a new string

```
name = "  ada "
name.strip().upper()   # thrown away
print(name)
  ada
```

Nothing happens

```
name = "  ada "
name = name.strip().upper()
print(name)
ADA
```

Keep the result

- Strings cannot change, so methods cannot change them either
- Chaining works left to right: strip first, then upper
- Normalise once, early: answer = answer.strip().lower() — then compare

STRING METHODS

split and join: text ⇄ list

```
>>> "Ada Byron Lovelace".split()
['Ada', 'Byron', 'Lovelace']

>>> "2026-11-02".split("-")
['2026', '11', '02']

>>> ".".join(["ada", "lovelace"])
'ada.lovelace'

>>> "/".join(["02", "11", "2026"])
'02/11/2026'
```

`split()`

without argument: any run of spaces, ends ignored

`split("-")`

cuts exactly at every -

`sep.join(list)`

the separator first, the list in the brackets

The list of pieces is a normal list: parts[0] is the first word, parts[-1] the last, len(parts) the number of words.

EXERCISE B ⏱ 30 min · notebook lab05-exercises.ipynb

B1 core Ten method calls on " Hello, Lab! ": predict, then run.

B2 core Survey answers "Yes", " yes ", "YES!", "No." …: clean each one, count yes / no / other.

B3 core Full name → ada.lovelace@studenti.unimi.it. Test with a middle name and with extra spaces.

B4 core split a sentence into words, join with -; reformat 2026-11-02 as 02/11/2026.

B5 stretch Password checker: print every rule it breaks (length, digit, uppercase,..).

B6 challenge Caesar cipher with ord() and chr(): shift every letter by 3, then decode.

Clean first, compare second: text.strip().lower()

03

Lists, a closer look

Two names for one list, more methods, and how not to lose elements

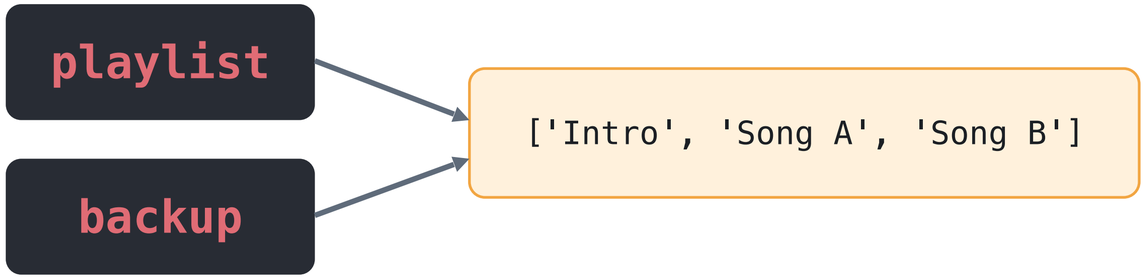
ALIASING

Two names, one list

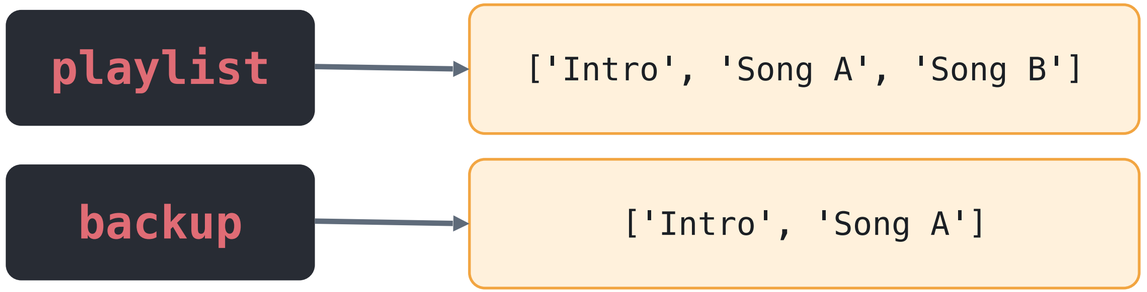
```
playlist = ["Intro", "Song A"]
backup = playlist
playlist.append("Song B")
print(backup)
['Intro', 'Song A', 'Song B']
```

= never copies. It attaches another name. A real copy: backup = playlist.copy() or playlist[:]. Check with backup is playlist.

backup = playlist



backup = playlist.copy()



More list methods

`q.insert(0, x)`	put x at position 0, shift the rest
`q.pop()`	remove and return the last element
`q.pop(0)`	remove and return the first
`q.extend(other)`	add every element of other
`q.index(x)`	position of x — ValueError if missing
`q.count(x)`	how often x appears
`q.reverse()`	reverse in place

```
queue = ["Ada", "Bob"]
queue.insert(0, "Dan")
first = queue.pop(0)
print(first, queue)
Dan ['Ada', 'Bob']
```

Unlike string methods, these change the list itself. Most return None — so queue = queue.reverse() loses your list.

Do not remove while you loop

```
readings = [21.5, -1, 22.0, -1, -1, 23.1]
for x in readings:
    if x == -1:
        readings.remove(x)
print(readings)
[21.5, 22.0, -1, 23.1]
```

No error — and one -1 survives

Removing shifts everything after it ***one place left,*** while the loop moves on: the element right after a removed one is skipped.

```
clean = []
for x in readings:
    if x != -1:
        clean.append(x)
print(clean)
[21.5, 22.0, 23.1]
```

Build a new list instead

Loop over one list, collect into another. The list you loop over never changes under your feet.

break and continue

```
for x in readings:
    if x == 999:
        break          # leave the loop
    if x == -1:
        continue       # skip to the next x
    total += x
    count += 1
```

break — stop the loop now; continue after it

continue — skip the rest of this round, take the next element

pass — do nothing — a placeholder where Python needs a line

Both also work in while loops. while True: … break is the usual shape of a program that runs until the user says quit — Part E.

EXERCISE C

⏱ 25 min · notebook lab05-exercises.ipynb

C1	core	backup = playlist, then append: what do both print? Make a real copy two ways
C2	core	The help queue: follow the list through append, insert, pop(0), extend, index, pop(), reverse.
C3	core	Remove every -1 from sensor readings: three attempts — predict, run, explain each.
C4	core	Average of valid readings: continue past -1, break at 999.
C5	stretch	Merge two lists of IDs into one sorted list without duplicates — loop and not in, no sets.
C6	challenge	Rotate a list by k positions with slicing; make k = 0, 5 and 7 work.

In C2, write the list down after every line before you run it.

04

Tuples and sets

A record that cannot change, and a collection without duplicates

Three containers

list

```
[21.5, 22.0]
```

- ordered, indices
- can change
- duplicates allowed

a collection that grows or shrinks

tuple

```
(45.46, 9.19)
```

- ordered, indices
- cannot change
- duplicates allowed

a fixed record: a point, a pair

set

```
{"Ada", "Bob"}
```

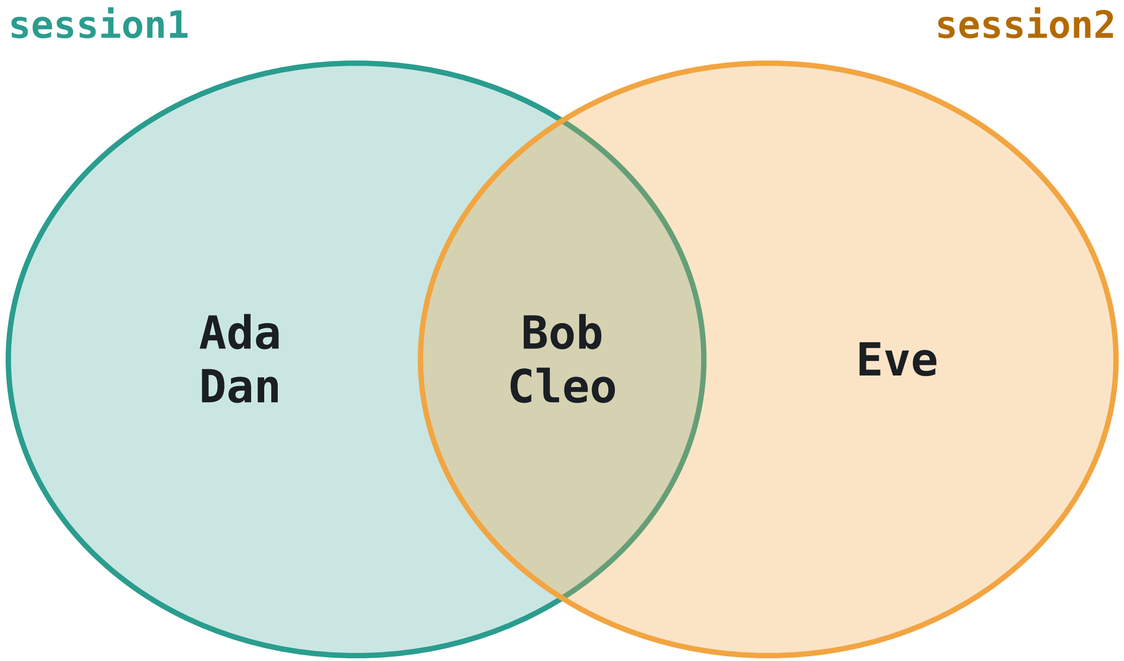
- no order, no indices
- can change
- never duplicates

membership, uniqueness, comparing groups

```
lat, lon = (45.46, 9.19)          # unpacking
for name, mark in [("Ada", 28), ("Bob", 17)]:
```

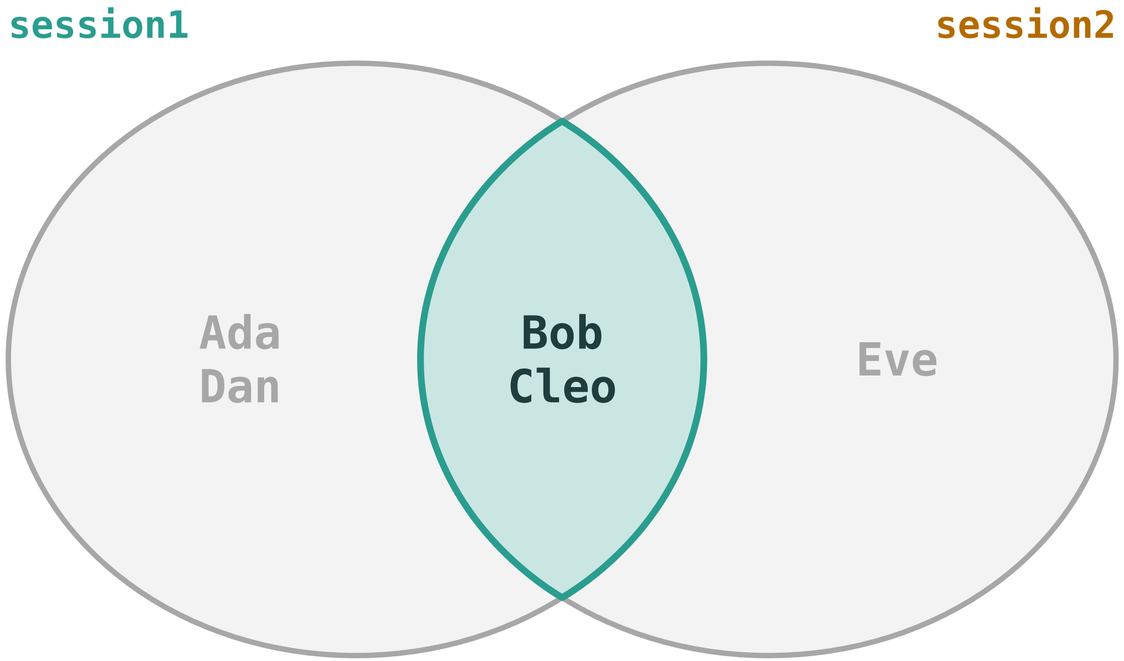
SETS

Comparing groups with sets



`len(set(names)) < len(names)` means there are duplicates. add() and discard() change a set; remove() raises KeyError if the element is missing.

Comparing groups with sets

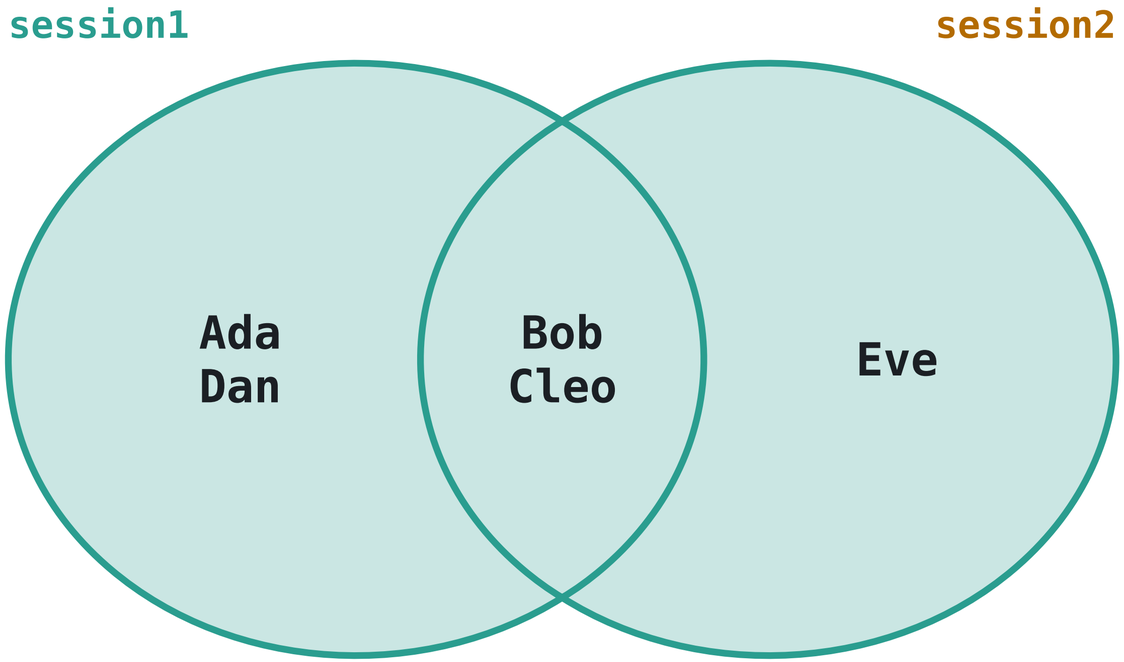


`s1 & s2`	in both	`Bob, Cleo`

`len(set(names)) < len(names)` means there are duplicates. add() and discard() change a set; remove() raises KeyError if the element is missing.

SETS

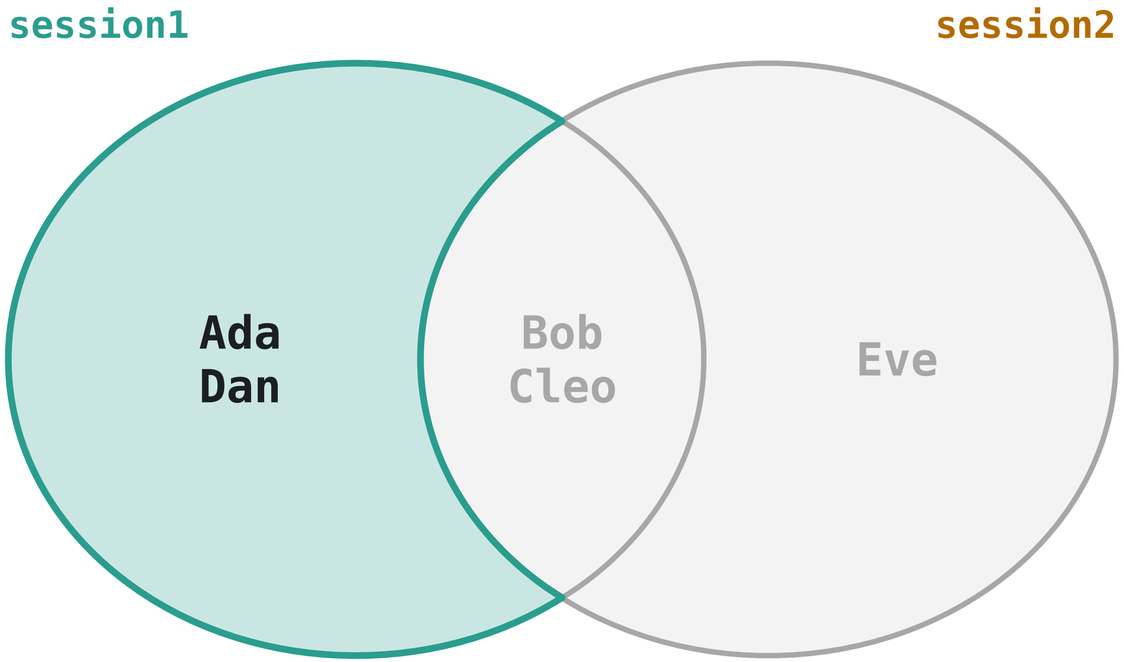
Comparing groups with sets



`s1 & s2`	in both	`Bob, Cleo`
`s1 | s2`	in at least one	`all five`

`len(set(names)) < len(names)` means there are duplicates. add() and discard() change a set; remove() raises KeyError if the element is missing.

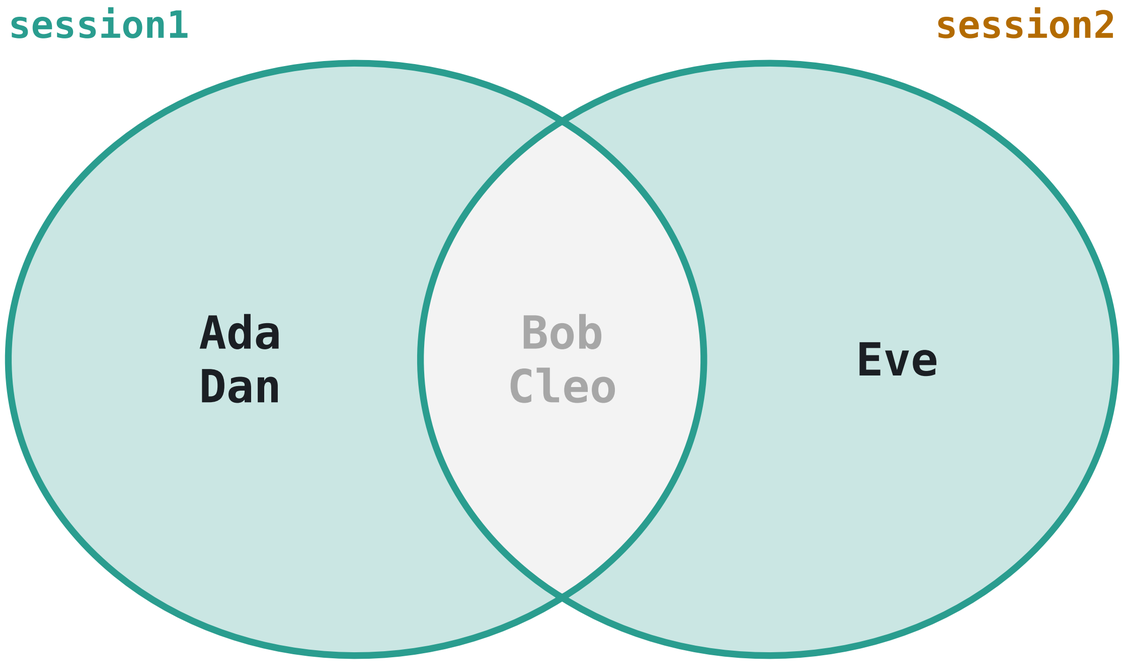
Comparing groups with sets



`s1 & s2`	in both	`Bob, Cleo`
`s1 | s2`	in at least one	`all five`
`s1 - s2`	only in s1	`Ada, Dan`

`len(set(names)) < len(names)` means there are duplicates. add() and discard() change a set; remove() raises KeyError if the element is missing.

Comparing groups with sets



`s1 & s2`	in both	`Bob, Cleo`
`s1 | s2`	in at least one	`all five`
`s1 - s2`	only in s1	`Ada, Dan`
`s1 ^ s2`	in exactly one	`Ada, Dan, Eve`

`len(set(names)) < len(names)` means there are duplicates. add() and discard() change a set; remove() raises KeyError if the element is missing.

EXERCISE D ⏱ 23 min · notebook lab05-exercises.ipynb

D1	core	milan = (45.4642, 9.19): unpack it, try to change it. type((5)) vs type((5,))?
D2	core	for name, mark in results: print passed / failed per student, then how many passed.
D3	core	Attendance sets: both, at least one, only session 1, exactly one. remove vs discard.
D4	core	Duplicate e-mails: len(set(...)) before and after cleaning the addresses.
D5	stretch	Common letters of two words, sorted, ignoring case and spaces.
D6	challenge	ids[0] on a set, a set of lists, a set of tuples: read the errors, explain.

Sets print in no particular order — use sorted(...) when the order matters.

05

todo.py

A tiny command interpreter: strings, lists and loops in one script

MINI-PROJECT

A program that understands four commands

```
$ python3 todo.py
> add read chapter 3
Added: read chapter 3
> add exercise sheet
Added: exercise sheet
> list
1. read chapter 3
2. exercise sheet
> done read chapter 3
Done: read chapter 3
> quit
Bye — 1 task left.
```

The shape of it

```
tasks = []
while True:
    line = input("> ").strip()
    parts = line.split(" ", 1)
    command = parts[0].lower()
    if command == "quit":
        break
    ...
```

Like a chatbot with a vocabulary of four words — and no learning at all.

EXERCISE E

⏱ 30 min · notebook lab05-exercises.ipynb

E1 core — todo.py in lab05: add TEXT, done TEXT, list (numbered from 1), quit. Tasks in a list, loop with while True.

E2 stretch — Empty line: nothing. Unknown command: short help. No duplicate tasks, ignoring upper/lower case.

E3 challenge — done 2 removes the second task: isdigit(), and numbers out of range.

Run it from the terminal: python3 todo.py (Windows: python todo.py).

Today you…

- sliced strings and built new ones
- cleaned, split and joined text
- saw two names share one list — and copied it
- removed elements without losing any
- used tuples for records and sets for groups
- wrote a small command interpreter

Next · sessions 6–9
with Fatme Hachem: dictionaries, functions, files and libraries.

Nothing to hand in. Keep todo.py — dictionaries will make it better.